Генеративные модели

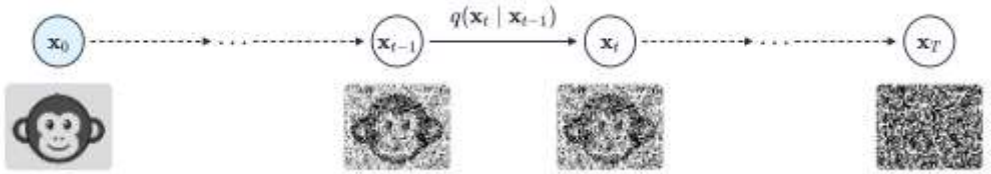
Лекция 8: $LDM, Guidance$

Повторение

Forward Diffusion Process

Определим ***прямой диффузионный процесс*** как ***марковскую цепь,*** в которой мы добавляем шум к нашим данным:

$$q(\mathbf{x}_{0:T}) = q(\mathbf{x}_0)\prod_{t=1}^{T} q(\mathbf{x}_t|\mathbf{x}_{t-1})$$



- $q(\mathbf{x}_0)$ – распределение реальных данных
- $q(\mathbf{x}_t|\mathbf{x}_{t-1})$ – переходное распределение

Для описания добавления шума на каждом шаге t будем использовать ***расписание шума*** (***noise schedule***) β_t:

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}\left(\mathbf{x}_t\middle|\sqrt{1-\beta_t}\cdot\mathbf{x}_{t-1}, \beta_t\boldsymbol{I}\right) \quad \Leftrightarrow \quad \mathbf{x}_t = \sqrt{1-\beta_t}\cdot\mathbf{x}_{t-1} + \sqrt{\beta_t}\cdot\boldsymbol{\epsilon}_t, \qquad \boldsymbol{\epsilon}_t \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$$

- $\sqrt{1-\beta_t}\cdot\mathbf{x}_{t-1}$ – среднее значение
- $\beta_t\boldsymbol{I}$ – ковариационная матрица
- β_t – малые числа, растущие со временем

Зашумление за один шаг

- Можно ли сразу узнать зашумленное состояние $\mathbf{x}_t$, если нам известно начальное состояние $\mathbf{x}_0$?

Утверждение:

- Пусть прямой процесс задан переходами:

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t|\sqrt{\alpha_t}\cdot\mathbf{x}_{t-1}, (1-\alpha_t)\boldsymbol{I})$$

- Тогда условное распределение $q(\mathbf{x}_t|\mathbf{x}_0)$, которое устанавливает связь между начальным состоянием $\mathbf{x}_0$ и любым последующим $\mathbf{x}_t$, имеет вид:

$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}\left(\mathbf{x}_t\middle|\sqrt{\overline{\alpha}_t}\cdot\mathbf{x}_0, (1-\overline{\alpha}_t)\boldsymbol{I}\right), \qquad \mathbf{x}_t = \sqrt{\overline{\alpha}_t}\cdot\mathbf{x}_0 + \sqrt{1-\overline{\alpha}_t}\cdot\boldsymbol{\epsilon}$$

где $\overline{\alpha}_t = \prod_{i=1}^{t}\alpha_i$, $\boldsymbol{\epsilon}-$ суммарный шум

Модель обратного перехода

- Чем меньше шаг, тем точнее обратный переход описывается гауссианой
- Поэтому каждый шаг модели тоже зададим гауссовым, а его параметры будет предсказывать нейросеть по $\mathbf{x}_t$ и t:

$$p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}\left(\boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t), \boldsymbol{\sigma}^2_{\boldsymbol{\theta},t}(\mathbf{x}_t)\right)$$

- Одна и та же сеть с параметрами $\boldsymbol{\theta}$ будет обрабатывать все шаги, номер t будет подаваться ей на вход

Итог:

- Мы выбрали гауссовскую форму модели, теперь нужно найти способ обучать её параметры, не вычисляя истинный обратный проход

Условное ядро

Мы получили:

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1}|\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0), \tilde{\beta}_t \cdot \boldsymbol{I})$$

Параметры нового распределения:

$$\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) = \frac{\sqrt{\alpha_t}(1-\overline{\alpha}_{t-1})}{1-\overline{\alpha}_t} \cdot \mathbf{x}_t + \frac{\sqrt{\overline{\alpha}_{t-1}}(1-\alpha_t)}{1-\overline{\alpha}_t} \cdot \mathbf{x}_0$$

$$\tilde{\beta}_t = \frac{(1-\alpha_t)(1-\overline{\alpha}_{t-1})}{1-\overline{\alpha}_t}$$

- При обучении $\mathbf{x}_0$ известно, поэтому $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$ можно вычислить и использовать как цель обучения для $p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t)$

Слагаемые $ELBO$

$$\mathcal{L}_{\boldsymbol{\theta}}(\mathbf{x}) = + \underbrace{\mathbb{E}_{q(\mathbf{x}_1|\mathbf{x}_0)} \log p_{\boldsymbol{\theta}}(\mathbf{x}_0|\mathbf{x}_1)}_{\mathcal{L}_0} - \underbrace{KL(q(\mathbf{x}_T|\mathbf{x}_0)||p(\mathbf{x}_T))}_{\mathcal{L}_T} - \sum_{t=2}^{T} \underbrace{\mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} KL(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)||p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t)}_{\mathcal{L}_{t-1}}$$

$\mathcal{L}_0$ – последний шаг обратной цепи, декодер $p_{\boldsymbol{\theta}}(\mathbf{x}_0|\mathbf{x}_1)$, отвечает за восстановление изображения $\mathbf{x}_0$ из $\mathbf{x}_1$:

$$\log p_{\boldsymbol{\theta}}(\mathbf{x}_0|\mathbf{x}_1) = \log \mathcal{N}\left(\mathbf{x}_0|\boldsymbol{\mu}_{\boldsymbol{\theta},1}(\mathbf{x}_1), \boldsymbol{\sigma}^2_{\boldsymbol{\theta},1}(\mathbf{x}_1)\right), \qquad \mathbf{x}_1 \sim q(\mathbf{x}_1|\mathbf{x}_0)$$

$\mathcal{L}_T$ – KL-дивергенция между двумя необучаемыми распределениями:

$$KL(q(\mathbf{x}_T|\mathbf{x}_0)||p(\mathbf{x}_T)) = KL\left(\mathcal{N}\left(\sqrt{\overline{\alpha}_T} \cdot \mathbf{x}_0, (1-\overline{\alpha}_T)\boldsymbol{I}\right)||\mathcal{N}(\mathbf{0}, \boldsymbol{I})\right) = const(\boldsymbol{\theta})$$

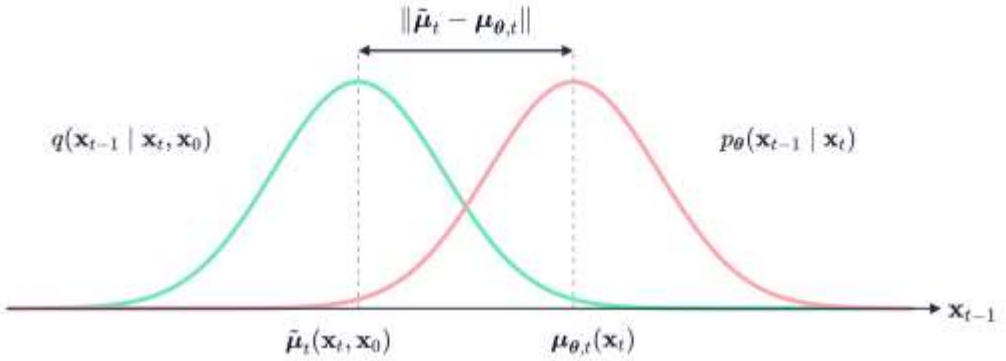
$\mathcal{L}_{t-1}$ – потеря на каждом шаге t

Каждый член заставляет нейросеть $p_{\boldsymbol{\theta}}$ быть похожей на истинное распределение q

Обучение среднего

Условное ядро и переход модели имеют одинаковую дисперсию $\tilde{\beta}_t$, поэтому:

$$KL\left(\mathcal{N}\left(\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0), \tilde{\beta}_t \boldsymbol{I}\right) \| \mathcal{N}\left(\boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t), \tilde{\beta}_t \boldsymbol{I}\right)\right)$$
$$= \frac{\left\|\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) - \boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t)\right\|^2}{2\tilde{\beta}_t}$$



- При одинаковой дисперсии и фиксированном t минимизация KL равносильна минимизации MSE между целевым и предсказанным средним:

$$\mathcal{L}_{t-1} = \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} KL\left(\mathcal{N}\left(\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0), \tilde{\beta}_t \boldsymbol{I}\right) || \mathcal{N}\left(\boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t), \tilde{\beta}_t \cdot \boldsymbol{I}\right)\right) = \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)}\left[\frac{1}{2\tilde{\beta}_t}\left|\left|\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) - \boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t)\right|\right|^2\right]$$

Reparameterization

$$\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) = \frac{1}{\sqrt{\alpha_t}} \cdot \mathbf{x}_t - \frac{(1-\alpha_t)}{\sqrt{\alpha_t(1-\overline{\alpha}_t)}} \cdot \boldsymbol{\epsilon}$$

Сделаем репараметризацию и зададим среднее в такой же форме:

$$\boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t) = \frac{1}{\sqrt{\alpha_t}} \cdot \mathbf{x}_t - \frac{(1-\alpha_t)}{\sqrt{\alpha_t(1-\overline{\alpha}_t)}} \cdot \boldsymbol{\epsilon}_{\boldsymbol{\theta},\boldsymbol{t}}(\mathbf{x}_t)$$

Теперь наша модель будет предсказывать шум $\boldsymbol{\epsilon}_{\boldsymbol{\theta},\boldsymbol{t}}(\mathbf{x}_t)$, вместо среднего $\boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t)$:

$$\mathcal{L}_{t-1} = \mathbb{E}_{\boldsymbol{\epsilon}\sim\mathcal{N}(\mathbf{0},\boldsymbol{I})}\left[\frac{(1-\alpha_t)^2}{2\tilde{\beta}_t\alpha_t(1-\overline{\alpha}_t)}\left|\left|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta},\boldsymbol{t}}(\mathbf{x}_t)\right|\right|^2\right] =$$

$$\mathbb{E}_{\boldsymbol{\epsilon}\sim\mathcal{N}(\mathbf{0},\boldsymbol{I})}\left[\frac{(1-\alpha_t)^2}{2\tilde{\beta}_t\alpha_t(1-\overline{\alpha}_t)}\left|\left|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta},\boldsymbol{t}}(\sqrt{\overline{\alpha}_t} \cdot \mathbf{x}_0 + \sqrt{1-\overline{\alpha}_t} \cdot \boldsymbol{\epsilon})\right|\right|^2\right]$$

Функция потерь $DDPM$

$$\mathcal{L}_{\boldsymbol{\theta}}(\mathbf{x}) = \mathbb{E}_{q(\mathbf{x}_1|\mathbf{x}_0)} \log p_{\boldsymbol{\theta}}(\mathbf{x}_0|\mathbf{x}_1) - KL(q(\mathbf{x}_T|\mathbf{x}_0)||p(\mathbf{x}_T)) - \sum_{t=2}^{T} \underbrace{\mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} KL(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)||p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t))}_{\mathcal{L}_{t-1}}$$

$$\mathcal{L}_{t-1} = \mathbb{E}_{\boldsymbol{\epsilon}\sim\mathcal{N}(\mathbf{0},\boldsymbol{I})} \left[\frac{(1-\alpha_t)^2}{2\tilde{\beta}_t \alpha_t (1-\overline{\alpha}_t)} \left|\left| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta},\boldsymbol{t}}(\sqrt{\overline{\alpha}_t} \cdot \mathbf{x}_0 + \sqrt{1-\overline{\alpha}_t} \cdot \boldsymbol{\epsilon}) \right|\right|^2 \right]$$

Отбросим весовой коэффициент:

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{\cdot\mathbf{x}_0\sim p_{data}} \mathbb{E}_{t\sim U(1,T)} \mathbb{E}_{\boldsymbol{\epsilon}\sim\mathcal{N}(\mathbf{0},\boldsymbol{I})} \left[\left|\left| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta},\boldsymbol{t}}(\sqrt{\overline{\alpha}_t} \cdot \mathbf{x}_0 + \sqrt{1-\overline{\alpha}_t} \cdot \boldsymbol{\epsilon}) \right|\right|^2 \right]$$

DDPM Training

Обучение:

- Берем объект $\mathbf{x}_0 \sim p_{data}(\mathbf{x})$
- Выбираем номер шага $t \sim U\{1, \dots, T\}$ и шум $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$
- Получаем зашумленный объект $\mathbf{x}_t = \sqrt{\overline{\alpha}_t} \cdot \mathbf{x}_0 + \sqrt{1 - \overline{\alpha}_t} \cdot \boldsymbol{\epsilon}$
- Подаем $\mathbf{x}_t$ и t в нейросеть
- Вычисляем MSE между реальным $\boldsymbol{\epsilon}$ и предсказанным шумом $\boldsymbol{\epsilon}_{\boldsymbol{\theta},\boldsymbol{t}}$
- Делаем шаг градиентного спуска и повторяем

DDPM Inference

Сэмплинг:

- Сэмплируем $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$
- Делаем обратный проход $t = T, \dots, 1$:

  Подаем $\mathbf{x}_t$ и t в обученную нейросеть**,** получаем $\boldsymbol{\epsilon}_{\boldsymbol{\theta},t}(\mathbf{x}_t)$

  Вычисляем среднее*:*

$$\boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t) = \frac{1}{\sqrt{\alpha_t}} \cdot \mathbf{x}_t - \frac{(1-\alpha_t)}{\sqrt{\alpha_t(1-\overline{\alpha}_t)}} \cdot \boldsymbol{\epsilon}_{\boldsymbol{\theta},t}(\mathbf{x}_t)$$

  Сэмплируем $\mathbf{x}_{t-1}$ из обратного процесса $p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t)$:

$$\mathbf{x}_{t-1} = \boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t) + \sqrt{\tilde{\beta}_t} \cdot \mathbf{z}, \qquad \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$$

План

- $DDPM$ и $NCSN$
- $Latent\ Diffusion\ Models$
- $Guidance$
- Оценка генеративных моделей

$DDPM$ и $NCSN$

DDPM и *NCSN*

Функция потерь *DDPM*:

$$\mathcal{L}_t = \mathbb{E}_{\boldsymbol{\epsilon}\sim\mathcal{N}(\mathbf{0},\boldsymbol{I})}\left[C_t^1 \left|\left|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta},\boldsymbol{t}}(\mathbf{x}_t)\right|\right|^2\right] = \mathbb{E}_{\boldsymbol{\epsilon}\sim\mathcal{N}(\mathbf{0},\boldsymbol{I})}\left[C_t^2 \left|\left|\frac{\boldsymbol{\epsilon}}{\sqrt{1-\overline{\alpha}_t}} - \frac{\boldsymbol{\epsilon}_{\boldsymbol{\theta},\boldsymbol{t}}(\mathbf{x}_t)}{\sqrt{1-\overline{\alpha}_t}}\right|\right|^2\right]$$

Найдем *score*-функцию для зашумления:

$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}\left(\mathbf{x}_t\middle|\sqrt{\overline{\alpha}_t}\cdot\mathbf{x}_0, (1-\overline{\alpha}_t)\boldsymbol{I}\right), \qquad \nabla_{\mathbf{x}_t}\log q(\mathbf{x}_t|\mathbf{x}_0) = -\frac{\mathbf{x}_t - \sqrt{\overline{\alpha}_t}\mathbf{x}_0}{1-\overline{\alpha}_t} = -\frac{\boldsymbol{\epsilon}}{\sqrt{1-\overline{\alpha}_t}}$$

Перепараметризуем выход сети как предсказание скора:

$$s_{\boldsymbol{\theta}}(\mathbf{x}_t, t) = -\frac{\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathbf{x}_t, t)}{\sqrt{1-\overline{\alpha}_t}}$$

Обучение *DDPM* и *NCSN*

Функцию потерь *DDPM* можно переписать через *score*-функцию:

$$\mathcal{L}_t = \mathbb{E}_{\boldsymbol{\epsilon}\sim\mathcal{N}(\mathbf{0},\boldsymbol{I})}\left[C_t^1 \left|\left|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta},\boldsymbol{t}}(\mathbf{x}_t)\right|\right|^2\right] = \mathbb{E}_{\boldsymbol{\epsilon}\sim\mathcal{N}(\mathbf{0},\boldsymbol{I})}\left[C_t^2 \left|\left|\boldsymbol{s}_{\boldsymbol{\theta},\boldsymbol{t}}(\mathbf{x}_t) - \nabla_{\mathbf{x}_t}\log q_t(\mathbf{x}_t|\mathbf{x}_0)\right|\right|^2\right]$$

Получили взвешенный *denoising score matching*

Различаются способы зашумления:

$$DDPM: \quad \mathbf{x}_t = \sqrt{\overline{\alpha}_t}\cdot\mathbf{x}_0 + \sqrt{1-\overline{\alpha}_t}\cdot\boldsymbol{\epsilon}$$

$$NCSN: \quad \mathbf{x}_t = \mathbf{x}_0 + \sigma_t\cdot\boldsymbol{\epsilon}$$

Обе модели учатся оценивать *score*-функцию на разных уровнях шума

Генерация $DDPM$ и $NCSN$

DDPM – Ancestral sampling:

- Начинаем с $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$
- Для $t = T, \dots, 1$ выполняем обратный проход:

$$\mathbf{x}_{t-1} = \boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t) + \sqrt{\tilde{\beta}_t} \cdot \boldsymbol{\epsilon} = \frac{1}{\sqrt{\alpha_t}} \cdot \mathbf{x}_t - \frac{(1-\alpha_t)}{\sqrt{\alpha_t(1-\overline{\alpha}_t)}} \cdot \boldsymbol{\epsilon}_{\boldsymbol{\theta},\boldsymbol{t}}(\mathbf{x}_t) + \sqrt{\tilde{\beta}_t} \cdot \boldsymbol{\epsilon} =$$

$$= \frac{1}{\sqrt{1-\beta_t}} \cdot \mathbf{x}_t + \frac{\beta_t}{\sqrt{1-\beta_t}} \cdot \boldsymbol{s}_{\boldsymbol{\theta},\boldsymbol{t}}(\mathbf{x}_t) + \sqrt{\tilde{\beta}_t} \cdot \boldsymbol{\epsilon}$$

NCSN – Anneal Langevin dynamics:

- Начинаем с $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \sigma_T^2 \boldsymbol{I})$
- На каждом уровне шума выполняем L шагов динамики Ланжевена:

$$\mathbf{x}_l = \mathbf{x}_{l-1} + \frac{\eta_t}{2} s_{\boldsymbol{\theta},\sigma_t}(\mathbf{x}_{l-1}) + \sqrt{\eta_t}\boldsymbol{\epsilon}_l$$

DDPM и *NCSN*

Итог 1:

- Обе модели оценивают скор зашумленных данных
- *DDPM* уменьшает исходный сигнал и добавляет шум, *NCSN* добавляет шум без уменьшения сигнала

$$DDPM: \quad \mathbf{x}_t = \sqrt{\overline{\alpha}_t} \cdot \mathbf{x}_0 + \sqrt{1 - \overline{\alpha}_t} \cdot \boldsymbol{\epsilon}$$

$$NCSN: \quad \mathbf{x}_t = \mathbf{x}_0 + \sigma_t \cdot \boldsymbol{\epsilon}$$

Итог 2:

- При генерации *DDPM* после одного обратного перехода переходим к следующему уровню шума
- При генерации *NCSN* выполняем несколько шагов динамики Ланжевена на одном уровне, затем уменьшаем шум

- Общая задача обеих моделей – научиться оценивать скор

- Способ зашумления и алгоритм генерации определяют, как использовать эту оценку для получения изображения

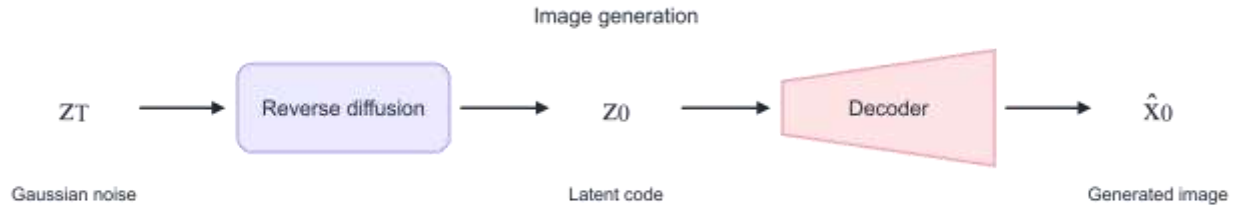
Latent Diffusion Models

Latent Diffusion Models

- В *DDPM* сеть обрабатывает изображение на каждом обратном шаге
- При увеличении разрешения растут расходы на память и количество вычислений

Идея:

- Сжать изображение в компактное представление и обучать на этом диффузию
- При генерации получаем скрытое представление с помощью диффузии и восстанавливаем изображение декодером



Основные вычисления происходят в скрытом пространстве, а декодер вызывается один раз в конце генерации

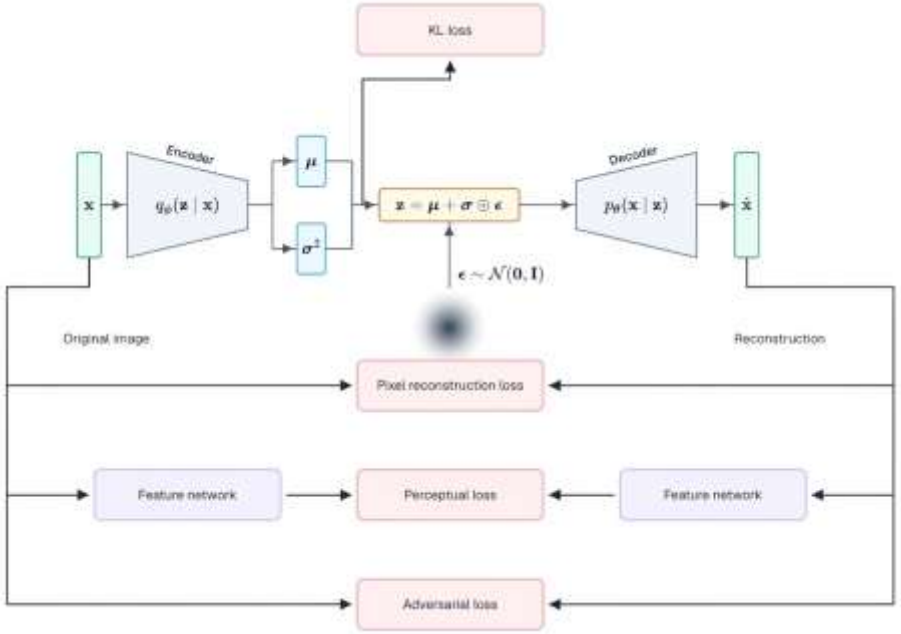
Этапы обучения LDM

Первый этап – обучение VAE:

- Кодировщик и декодировщик учатся восстанавливать изображения

В отличие от обычного VAE, используем новую функцию потерь:

$$\mathcal{L}_{VAE} = \mathcal{L}_{rec} + \lambda_{perc}\mathcal{L}_{perc} + \lambda_{adv}\mathcal{L}_{adv} + \lambda_{KL}\mathcal{L}_{KL}$$



Этапы обучения LDM

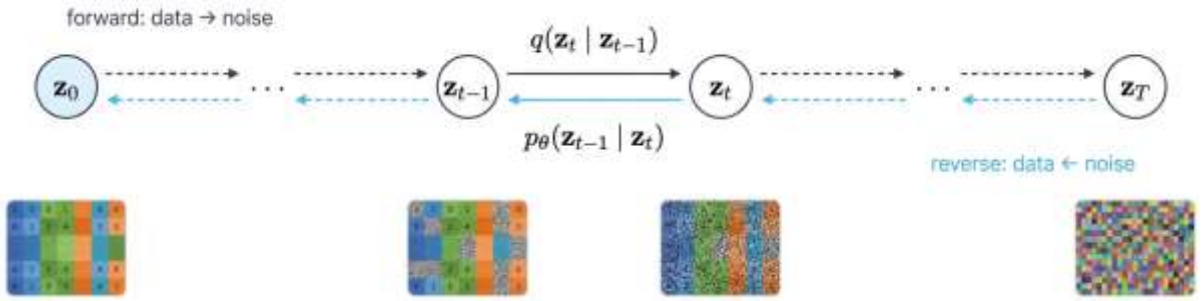
Второй этап – обучение диффузии:

- Кодируем все изображения из обучающей выборки и используем их для обучения

Обучаем сеть предсказывать добавленный шум в латентном пространстве:

$$\boldsymbol{z}_t = \sqrt{\overline{\alpha}_t} \cdot \boldsymbol{z}_0 + \sqrt{1 - \overline{\alpha}_t} \cdot \boldsymbol{\epsilon}$$

$$\mathcal{L}_{LDM} = \mathbb{E}_{\boldsymbol{z}_0,\, t,\, \boldsymbol{\epsilon}} \left[\left|\left| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta}, \boldsymbol{t}}(\boldsymbol{z}_t) \right|\right|^2 \right]$$



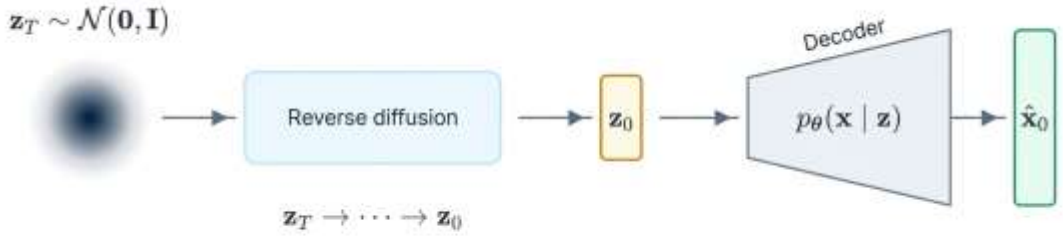
Генерация *LDM*

- Начинаем с гауссовского шума и выполняем обратные шаги диффузии в скрытом пространстве:

$$\boldsymbol{z}_T \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$$

$$\boldsymbol{z}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \cdot \boldsymbol{z}_t - \frac{(1-\alpha_t)}{\sqrt{\alpha_t(1-\overline{\alpha}_t)}} \cdot \boldsymbol{\epsilon}_{\boldsymbol{\theta},\boldsymbol{t}}(\boldsymbol{z}_t) + \sqrt{\tilde{\beta}_t} \cdot \boldsymbol{\epsilon}$$

- После завершения цепи передаём полученный $\boldsymbol{z}_0$ в декодер



Guidance

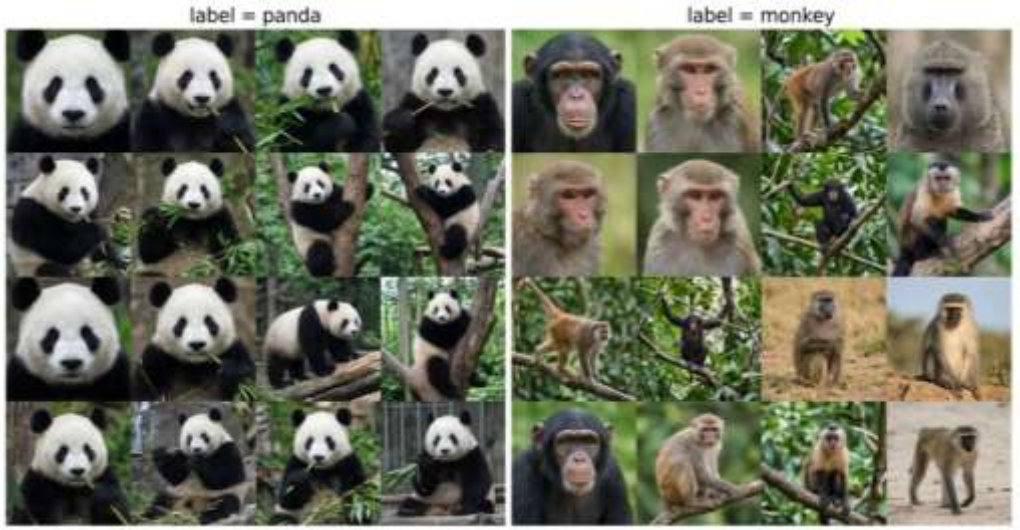
Условная генерация

- При безусловной генерации модель сама выбирает, какой объект появится на изображении

- Чтобы задать нужный класс, можно использовать метку y

- Условная диффузионная модель будет принимать зашумленное изображение $\mathbf{x}_t$, номер шага t и метку y:

$$\epsilon_{\theta,t}(\mathbf{x}_t, y)$$



- При генерации метка остается одинаковой на всех обратных шагах

Условная генерация

- Метка класса задаёт тип объекта
- Текстовое описание позволяет дополнительно указать действия, окружение и стиль изображения

- Теперь условие y — текстовый запрос
- Текстовый кодировщик превращает описание в вектор, который модель использует при предсказании шума

$$\epsilon_{\theta,t}(\mathbf{x}_t, \tau(y))$$

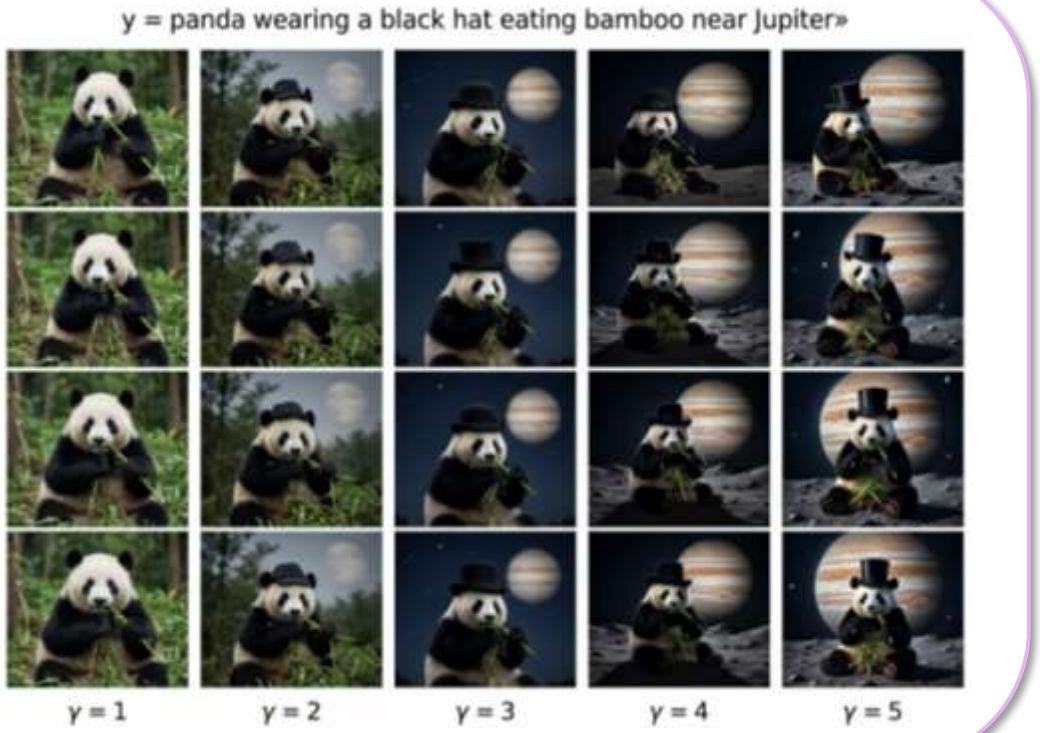
$\tau(y)$ — представление текста



Guidance

- Условная модель учитывает метку или описание, но результат не всегда хорошо соответствует заданию

- При генерации хотелось бы регулировать следование инструкции
- Введём коэффициент управления γ (***guidance scale***), который позволит усиливать влияние текста
- При небольшом γ модель допускает больше вариантов, при большем γ – сильнее подчеркивает признаки из описания, но уменьшает разнообразие



Classifier guidance

- Начнем с условия в виде метки класса y

- В обратном шаге *DDPM* используется безусловная *score*-функция:

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\Big(\mathbf{x}_t + \beta_t \cdot \underbrace{\boldsymbol{s}_{\boldsymbol{\theta},\boldsymbol{t}}(\mathbf{x}_t)}_{\approx \nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t)}\Big) + \sqrt{\tilde{\beta}_t} \cdot \boldsymbol{\epsilon}_t$$

- Для генерации выбранного класса нужно заменить её на *score*-функцию условного распределения:

$$\nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t|y)$$

- Нужно научиться оценивать *score*-функцию условного распределения

Classifier guidance

Свяжем условную и безусловную плотности через формулу Байеса:

$$q_t(\mathbf{x}_t|y) = \frac{q_t(\mathbf{x}_t) \cdot q_t(y|\mathbf{x}_t)}{q_t(y)}$$

Возьмём градиент логарифма по текущему состоянию $\mathbf{x}_t$:

$$\nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t|y) = \nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t) + \nabla_{\mathbf{x}_t} \log q_t(y|\mathbf{x}_t)$$

- Первое слагаемое уже оценивает диффузионная сеть
- Для второго обучим классификатор на зашумлённых изображениях:

$$p_{\boldsymbol{\phi}}(y|\mathbf{x}_t, t) \approx q_t(y|\mathbf{x}_t)$$

Classifier guidance

Получаем оценку условной *score*-функции:

$$\boldsymbol{s}_t(\mathbf{x}_t, y) = \boldsymbol{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t) + \nabla_{\mathbf{x}_t} \log p_{\boldsymbol{\phi}}(y|\mathbf{x}_t, t)$$

Градиент классификатора показывает, как изменить текущее состояние, чтобы увеличить вероятность выбранного класса

Чтобы регулировать влияние класса, умножим поправку на коэффициент γ:

$$\boldsymbol{s}_t^{\gamma}(\mathbf{x}_t, y) = \boldsymbol{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t) + \gamma \nabla_{\mathbf{x}_t} \log p_{\boldsymbol{\phi}}(y|\mathbf{x}_t, t)$$

- $\gamma = 0 -$ безусловная генерация
- $\gamma = 1 -$ условная *score*-функция
- $\gamma > 1 -$ дополнительное усиление влияния класса

Коэффициент γ выбираем при генерации, модель переобучать не нужно

Изменение распределения

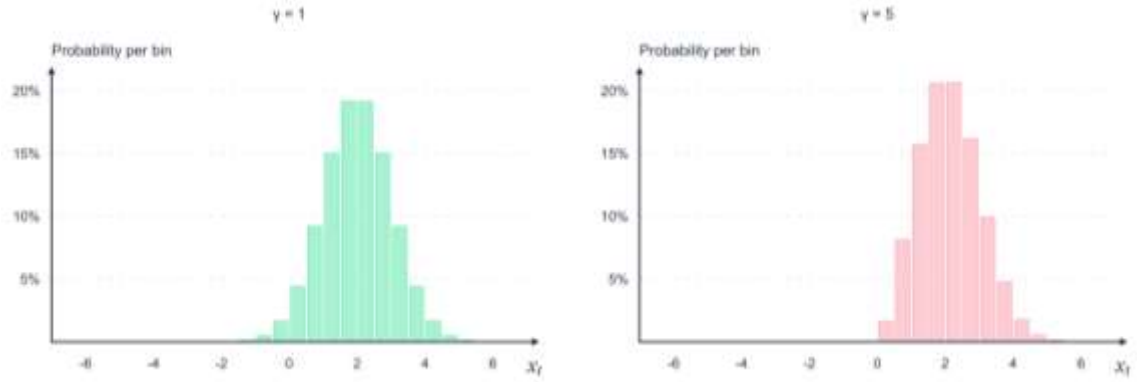
Для фиксированного уровня шума:

$$\boldsymbol{s}_t^{\gamma}(\mathbf{x}_t, y) = \nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t) + \gamma \nabla_{\mathbf{x}_t} \log q_t(y|\mathbf{x}_t) = \nabla_{\mathbf{x}_t} \log[q_t(\mathbf{x}_t) \cdot q_t(y|\mathbf{x}_t)^{\gamma}]$$

- *Guidance* меняет вес каждого состояния:

$$q_t^*(\mathbf{x}_t|y) \propto q_t(\mathbf{x}_t) \cdot q_t(y|\mathbf{x}_t)^{\gamma}$$

- Рост γ усиливает различие между весами
- После нормировки одни столбцы становятся выше за счёт уменьшения других



- Усиление *guidance* повышает предпочтение характерных признаков класса, но может уменьшать разнообразие

Алгоритм *Classifier guidance*

Обучение:

- Обучаем безусловную диффузионную модель
- Обучаем классификатор $p_{\boldsymbol{\phi}}(y|\mathbf{x}_t, t)$ на изображениях с разными уровнями шума

Генерация:

- Начинаем с $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$
- Для $t = T, \dots, 1$ вычисляем исправленный скор:

$$\boldsymbol{s}_t^{\gamma}(\mathbf{x}_t, y) = \boldsymbol{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t) + \gamma \nabla_{\mathbf{x}_t} \log p_{\boldsymbol{\phi}}(y|\mathbf{x}_t, t)$$

- Выполняем обратный переход:

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t + \beta_t \cdot \boldsymbol{s}_t^{\gamma}(\mathbf{x}_t, y)\right) + \sqrt{\tilde{\beta}_t} \cdot \boldsymbol{\epsilon}_t, \qquad \boldsymbol{\epsilon}_t \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$$

$Classifier - free\ guidance$

- В *classifier guidance* пришлось обучать дополнительный классификатор для зашумлённых изображений:

$$\nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t|y) = \nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t) + \nabla_{\mathbf{x}_t} \log q_t(y|\mathbf{x}_t)$$

- Попробуем выразить его вклад через условный и безусловный скоры

Из полученного вывода следует:

$$\nabla_{\mathbf{x}_t} \log q_t(y|\mathbf{x}_t) = \nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t|y) - \nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t)$$

- Нужное направление можно получить как разность условного и безусловного скоров

- Вместо вероятности класса будем оценивать два скора

$Classifier - free\ guidance$

Подставим найденную разность в формулу *guidance*:

$$\boldsymbol{s}_t^{\gamma}(\mathbf{x}_t, y) = \nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t) + \gamma \nabla_{\mathbf{x}_t} \log q_t(y|\mathbf{x}_t) = \nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t) + \gamma \left(\nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t|y) - \nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t) \right) =$$

$$= (1 - \gamma) \nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t) + \gamma \nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t|y)$$

- Можно обучать две сети: условную и безусловную
- При генерации объединить их ответы

- Обе сети будут решать задачу удаления шума – можно попытаться объединить всё в одну модель

$Classifier - free\ guidance$

- Введем пустое условие $\emptyset$, которое означает отсутствие метки или текста
- При обучении с вероятностью p_{drop} заменяем настоящее условие пустым:

$$\hat{y} = \begin{cases} \emptyset & \text{с вероятностью } p_{drop} \\ y & \text{с вероятностью } 1 - p_{drop} \end{cases}$$

Все остальное сохраняется и обучаем модель с обычной ошибкой предсказания шума:

$$\mathcal{L} = \mathbb{E}_{\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})} \left[\left|\left| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta}, \boldsymbol{t}}(\mathbf{x}_t, \hat{y}) \right|\right|^2 \right]$$

- Одна сеть учится работать с условием и без него
- p_{drop} выбираем при обучении, $\gamma -$ при генерации

Генерация $Classifier - free\ guidance$

- На каждом обратном шаге получаем от модели два ответа для одинаковых $\mathbf{x}_t$ и t:

$$\boldsymbol{\epsilon}_{cond} = \boldsymbol{\epsilon}_{\boldsymbol{\theta},\boldsymbol{t}}(\mathbf{x}_t, y), \qquad \boldsymbol{\epsilon}_{uncond} = \boldsymbol{\epsilon}_{\boldsymbol{\theta},\boldsymbol{t}}(\mathbf{x}_t, \emptyset)$$

- Усиливаем разность условного и безусловного предсказаний:

$$\widehat{\boldsymbol{\epsilon}}_t^{\,\gamma} = \boldsymbol{\epsilon}_{uncond} + \gamma(\boldsymbol{\epsilon}_{cond} - \boldsymbol{\epsilon}_{uncond})$$

- Подставляем новый шум в обратный переход диффузии:

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\overline{\alpha}_t}} \cdot \widehat{\boldsymbol{\epsilon}}_t^{\,\gamma}\right) + \sqrt{\tilde{\beta}_t} \cdot \boldsymbol{\epsilon}_t, \qquad \boldsymbol{\epsilon}_t \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$$

Оценка генеративных моделей

Требования к генеративным моделям

Качество изображений:

- Объекты должны выглядеть правдоподобно, без заметных искажений и артефактов



Разнообразие и покрытие изображений:

- Модель должна воспроизводить разные классы, формы, ракурсы и сцены



Соответствие условию:

При генерации по метке или тексту изображение должно следовать инструкции



Оценка генеративных моделей

- Нужно уметь оценивать отдельные изображения, весь набор генераций и следование заданию («инструкция – результат»)
- Для каждой задачи понадобятся отдельные метрики

- Для моделей с вычислимым правдоподобием можно оценивать вероятность тестовых данных

$$NLL = -\frac{1}{n}\sum_{i=1}^{n} \log p_\theta(\mathbf{x}_i)$$

- Для VAE и диффузионных моделей можем использовать вариационные оценки

- У GAN нет формулы для плотности
- Поэтому будем сравнивать модели по сгенерированным примерам

Frechet Inception Distance

Вспомним расстояние Вассерштейна в случае $s = 2$:

$$W_2^2(p, q) = \inf_{\gamma \in \Gamma(p,q)} \mathbb{E}_{(\mathbf{x},\boldsymbol{y})\sim\gamma}\left[\left|\left|\mathbf{x} - \boldsymbol{y}\right|\right|_2^2\right]$$

- Расстояние между пикселями плохо отражает сходство изображений (небольшой сдвиг может сильно изменить значение пикселей)
- Пропустим реальные и сгенерированные изображения через одну модель (Inception V3):

$$\mathbf{h} = f(\mathbf{x})$$

Для каждой выборки найдем среднее и дисперсию:

$$\boldsymbol{\mu} = \frac{1}{n}\sum_{i=1}^{n} \boldsymbol{h}_i\,, \qquad \boldsymbol{\Sigma} = \frac{1}{n-1}\sum_{i}^{n} (\boldsymbol{h}_i - \boldsymbol{\mu})(\boldsymbol{h}_i - \boldsymbol{\mu})^T$$

Frechet Inception Distance

Приблизим каждое распределение признаков гауссовским:

$$FID = \left|\left|\boldsymbol{\mu}_{data} - \boldsymbol{\mu}_{\boldsymbol{\theta}}\right|\right|_2^2 + tr\left[\boldsymbol{\Sigma}_{\text{data}} + \boldsymbol{\Sigma}_{\boldsymbol{\theta}} - 2\left(\boldsymbol{\Sigma}_{\text{data}}^{\frac{1}{2}}\boldsymbol{\Sigma}_{\boldsymbol{\theta}}\boldsymbol{\Sigma}_{\text{data}}^{\frac{1}{2}}\right)^{\frac{1}{2}}\right]$$

- Чем меньше FID, тем ближе среднее и ковариация признаков реальных изображений и генераций

Проблемы:

- Среднее и ковариация не описывают произвольное распределение целиком
- Может не коррелировать с человеческими оценками

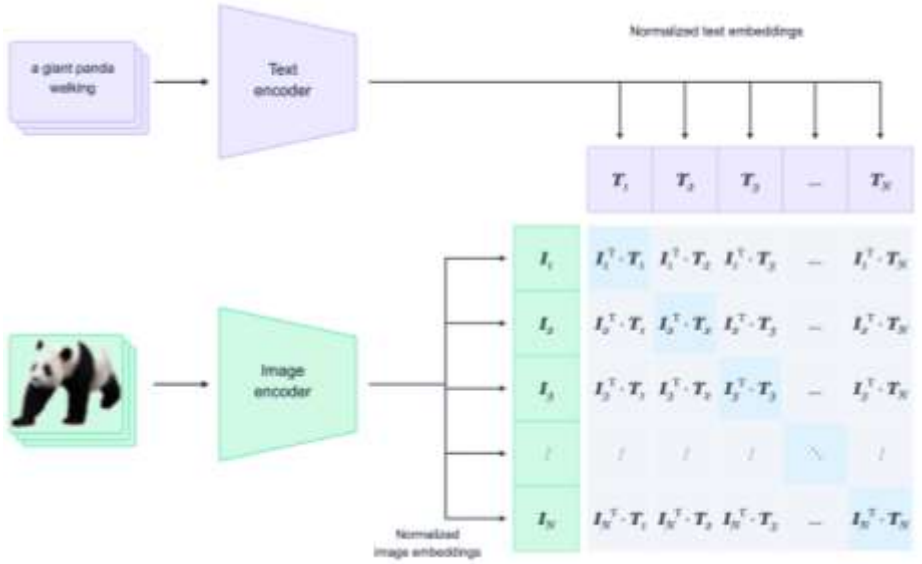
CLIP Score

Реалистичное изображение может не соответствовать запросу

CLIP содержит 2 предобученных кодировщика – для изображений $\mathbf{x}$ и текста y

Получаем их представления и нормируем на длину:

$$\mathbf{v} = \frac{f_{img}(\mathbf{x})}{||f_{img}(\mathbf{x})||_2}, \qquad \mathbf{c} = \frac{f_{text}(y)}{||f_{text}(y)||_2}$$



Сравниваем векторы по косинусному сходству:

$$sim(\mathbf{x}, y) = \mathbf{v}^{\mathrm{T}}\mathbf{c}$$

Human Evaluation

Автоматические метрики могут пропускать ошибки, заметные человеку

Side-by-side (***SBS***):

- Показываем разметчикам результаты двух моделей
- Можно раздельно оценивать соответствие запросу, дефекты и общее предпочтение



Model A Model B

SBS рассчитывается следующим образом:

$$SBS_A = \frac{n_A}{N}, \qquad SBS_B = \frac{n_B}{N}, \qquad SBS_{unknown} = \frac{n_{unknown}}{N}$$

где N – количество оценок, n_A – количество голосов за вариант A, n_B – количество голосов за вариант B, $n_{unknown}$ – количество голосов за вариант «не знаю»

Спасибо за внимание!